Day 15: Raft Consensus, Leader Election & Log Replication

Coordinating multiple nodes so they behave like one consistent state machine.

Why Consensus?

- Replication alone doesn't guarantee correctness.
- Consensus ensures a single, agreed order of operations.
- Prevents split-brain and conflicting committed history.

Raft Roles & Terms

Follower
Follows the leader and responds to RPCs.

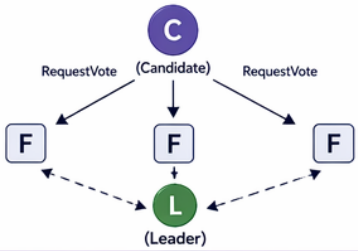
Candidate
Starts election, requests votes from peers.

Leader
Handles client requests, replicates log entries.

Time is divided into logical terms. At most one leader should be elected per term.

Leader Election (Majority Vote)

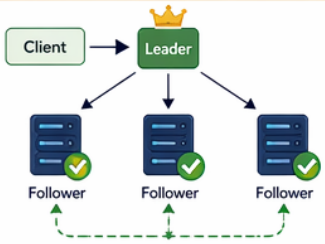
1. Candidate increments term and requests votes.
2. Each node votes once per term for an eligible candidate.
3. Candidate with majority votes becomes leader.



Majority = floor(N/2) + 1

Log Replication & Commit

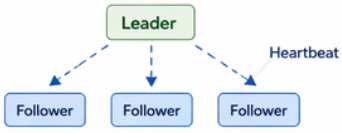
1. Client sends command to Leader.
2. Leader appends entry to its log.
3. Leader replicates entry to followers (AppendEntries).
4. Followers store the entry and acknowledge.
5. When Leader receives acknowledgements from a majority (including itself) in the current term, the entry is committed.



Committed entries are safe and will never be lost.

Heartbeats

- Leaders send heartbeats (empty AppendEntries) periodically.
- Prevents new elections and maintains leadership.



Versioned Logs

- Each log entry has: term, index, command.
- Entries with older terms or conflicting indexes are rejected.
- Ensures a consistent, ordered history across the cluster.

1	2	3	4	5
(t1)	(t1)	(t2)	(t2)	(t2)
cmd1	cmd2	cmd3	cmd4	cmd5

Safety & Trade-offs

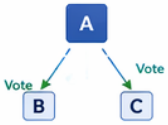
- Majority quorum is the core safety boundary in Raft.
- Minority partitions should not make progress.
- Safety first, availability may decrease during failures.
- Performance depends on quorum size, network latency, and batching strategy.

Raft Consensus Flow (Example with 3 Nodes)

1 Election Starts
Node A times out and starts election for term 5.

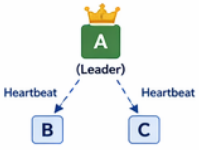
A → B : RequestVote (t5)
A → C : RequestVote (t5)

2 Majority Votes
Nodes B and C vote for A. A gets majority (2/3).



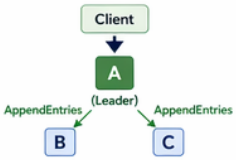
B → A : Vote Granted (t5)
C → A : Vote Granted (t5)

3 Leader Elected
Node A becomes Leader for term 5.



A sends heartbeats to maintain leadership.

4 Log Replication
Client sends command to Leader A.



Leader appends entry and replicates to followers.

5 Commit Entry
A gets acknowledgements from majority (A + B or A + C). Entry is committed.

Committed entries are applied to the state machine.

Interview Takeaways

- Consensus is about ordered agreement, not just copying data.
- Majority quorum is the core safety boundary in Raft.
- Terms prevent stale leaders from overwriting newer history.
- Leader-based replication simplifies reasoning about write order.
- Commit rules must be explicit, otherwise logs can still diverge.

Quick Reference

N	Total number of nodes
W	Write quorum (min nodes to accept write)
R	Read quorum (min nodes to serve read)
Majority	floor(N/2) + 1
Commit Rule	Entry is committed when stored on a majority in current term

Common Example (N = 3)

R	W	R + W > N	Result
1	1	2 > 3	No (weak)
2	1	3 > 3	Yes (good)
1	2	3 > 3	Yes (good)
2	2	4 > 3	Yes (strong)
3	2	5 > 3	Yes (strongest)

Raft gives us a clear, understandable way to achieve consensus and build reliable, fault-tolerant distributed systems.

Overview from README.md

Day 15: Raft Consensus, Leader Election, and Log Replication

Day 15 focuses on how distributed systems agree on one ordered history of updates despite failures.

It covers:
- consensus fundamentals
- Raft roles and terms
- leader election and majority voting
- replicated logs and commit rules
- heartbeats and leadership stability
- safety vs availability trade-offs under partitions

In short, Day 15 is about coordinating multiple nodes so they behave like one consistent state machine.

[Day 15 PDF](./System_Design_Day_15.pdf)

Core ideas

Why consensus matters
- Replication alone does not guarantee correctness when nodes disagree.
- A `consensus protocol` ensures a single, agreed order of operations.
- This prevents split-brain behavior and conflicting committed history.

Raft roles and terms
- Nodes move between `Follower`, `Candidate`, and `Leader` roles.
- Time is divided into logical `terms`.
- At most one leader should be elected per term by majority vote.

Leader election
- A candidate increments term and requests votes from peers.
- Each node votes once per term for an eligible candidate.
- Candidate with majority becomes leader; others follow that leader.

Log replication and commit
- Clients write through the leader.
- Leader appends an entry locally, then replicates to followers.
- Entry is committed when a majority acknowledges it in the leader term.

Safety and trade-offs
- Majority rules protect safety but can reduce availability during failures.
- Minority partitions should not make progress.
- Performance depends on quorum size, network latency, and batching strategy.

Day-15 sample: Raft Consensus (Simplified)

This repository includes a small Raft-style sample in both Python and Java.

Functional requirements
- Elect a leader via majority voting
- Reject stale leader/candidate terms
- Append commands through the elected leader

File: demo.py

```python
from raft_cluster import RaftCluster


def print_state(label: str, cluster: RaftCluster) -> None:
    print(label)
    for row in cluster.cluster_state():
        print(
            "  "
            + f"{row['node']} "
            + f"avail={row['available']} "
            + f"role={row['role']} "
            + f"term={row['term']} "
            + f"log={row['log_len']} "
            + f"commit={row['commit_index']}"
        )


def main() -> None:
    cluster = RaftCluster(["n1", "n2", "n3", "n4", "n5"])

    elected = cluster.elect_leader("n1")
    print("leader election for n1:", elected)
    print_state("cluster after election", cluster)

    committed = cluster.leader_append("set x=10")
    print("\nappend set x=10 committed:", committed)

    cluster.set_availability("n4", False)
    cluster.set_availability("n5", False)
    print_state("\ncluster after two follower failures", cluster)

    committed = cluster.leader_append("set y=20")
    print("\nappend set y=20 committed with 3/5 up:", committed)

    cluster.set_availability("n3", False)
    print_state("\ncluster after third failure", cluster)

    committed = cluster.leader_append("set z=30")
    print("\nappend set z=30 committed with 2/5 up:", committed)


if __name__ == "__main__":
    main()
```

File: raft_cluster.py

```
from __future__ import annotations


from dataclasses import dataclass
from enum import Enum


class Role(Enum):
    FOLLOWER = "FOLLOWER"
    CANDIDATE = "CANDIDATE"
    LEADER = "LEADER"


@dataclass(frozen=True)
class LogEntry:
    term: int
    command: str


class RaftNode:
    def __init__(self, node_id: str) -> None:
        self.node_id = node_id
        self.role = Role.FOLLOWER
        self.current_term = 0
        self.voted_for: str | None = None
        self.log: list[LogEntry] = []
        self.commit_index = -1
        self.available = True

    def request_vote(
        self,
        candidate_id: str,
        term: int,
        last_log_index: int,
        last_log_term: int,
    ) -> bool:
        if not self.available:
            return False

        if term < self.current_term:
            return False

        if term > self.current_term:
            self.current_term = term
            self.voted_for = None
            self.role = Role.FOLLOWER

        if self.voted_for is not None and self.voted_for != candidate_id:
            return False

        if not self._is_candidate_log_up_to_date(last_log_index, last_log_term):
```

```
            return False

        self.voted_for = candidate_id
        return True

    def append_entries(
        self,
        leader_term: int,
        prev_log_index: int,
        prev_log_term: int,
        entries: list[LogEntry],
        leader_commit: int,
    ) -> bool:
        if not self.available:
            return False

        if leader_term < self.current_term:
            return False

        self.current_term = leader_term
        self.role = Role.FOLLOWER

        if prev_log_index >= 0:
            if prev_log_index >= len(self.log):
                return False
            if self.log[prev_log_index].term != prev_log_term:
                return False

        insert_at = prev_log_index + 1
        self.log = self.log[:insert_at] + entries

        if leader_commit > self.commit_index:
            self.commit_index = min(leader_commit, len(self.log) - 1)

        return True

    def _is_candidate_log_up_to_date(self, last_log_index: int, last_log_term: int) -> bool:
        my_last_index = len(self.log) - 1
        my_last_term = self.log[my_last_index].term if my_last_index >= 0 else -1

        if last_log_term != my_last_term:
            return last_log_term > my_last_term
        return last_log_index >= my_last_index


class RaftCluster:
    def __init__(self, node_ids: list[str]) -> None:
        if len(node_ids) < 3:
            raise ValueError("cluster should have at least 3 nodes")

        self.nodes = [RaftNode(node_id) for node_id in node_ids]
        self._leader_id: str | None = None
```

```python

    def elect_leader(self, candidate_id: str) -> bool:
        candidate = self._node(candidate_id)
        if not candidate.available:
            return False

        candidate.role = Role.CANDIDATE
        candidate.current_term += 1
        candidate.voted_for = candidate.node_id
        term = candidate.current_term
        votes = 1

        last_log_index = len(candidate.log) - 1
        last_log_term = candidate.log[last_log_index].term if last_log_index >= 0 else -1

        for node in self.nodes:
            if node.node_id == candidate_id:
                continue
            if node.request_vote(candidate_id, term, last_log_index, last_log_term):
                votes += 1

        if votes >= self._majority():
            for node in self.nodes:
                node.role = Role.FOLLOWER
                if node.current_term < term:
                    node.current_term = term
            candidate.role = Role.LEADER
            self._leader_id = candidate_id
            return True

        candidate.role = Role.FOLLOWER
        return False

    def leader_append(self, command: str) -> bool:
        leader = self.leader()
        if leader is None or not leader.available:
            return False

        entry = LogEntry(term=leader.current_term, command=command)
        leader.log.append(entry)
        entry_index = len(leader.log) - 1
        prev_index = entry_index - 1
        prev_term = leader.log[prev_index].term if prev_index >= 0 else -1

        acknowledgements = 1
        for node in self.nodes:
            if node.node_id == leader.node_id:
                continue
            if node.append_entries(
                leader_term=leader.current_term,
                prev_log_index=prev_index,
                prev_log_term=prev_term,
```

```
                entries=[entry],
                leader_commit=leader.commit_index,
            ):
                acknowledgements += 1

        if acknowledgements >= self._majority():
            leader.commit_index = entry_index
            for node in self.nodes:
                if node.node_id == leader.node_id:
                    continue
                node.append_entries(
                    leader_term=leader.current_term,
                    prev_log_index=entry_index,
                    prev_log_term=entry.term,
                    entries=[],
                    leader_commit=leader.commit_index,
                )
            return True

        leader.log.pop()
        return False

    def leader(self) -> RaftNode | None:
        if self._leader_id is None:
            return None
        node = self._node(self._leader_id)
        if node.role != Role.LEADER:
            return None
        return node

    def set_availability(self, node_id: str, available: bool) -> None:
        node = self._node(node_id)
        node.available = available
        if node_id == self._leader_id and not available:
            self._leader_id = None

    def cluster_state(self) -> list[dict[str, str | int | bool]]:
        state: list[dict[str, str | int | bool]] = []
        for node in self.nodes:
            state.append(
                {
                    "node": node.node_id,
                    "available": node.available,
                    "role": node.role.value,
                    "term": node.current_term,
                    "log_len": len(node.log),
                    "commit_index": node.commit_index,
                }
            )
        return state

    def _node(self, node_id: str) -> RaftNode:
```

```
        for node in self.nodes:
            if node.node_id == node_id:
                return node
        raise KeyError(f"unknown node: {node_id}")

    def _majority(self) -> int:
        return (len(self.nodes) // 2) + 1
```

File: Main.java

```java
import java.util.List;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        RaftCluster cluster = new RaftCluster(List.of("n1", "n2", "n3", "n4", "n5"));

        boolean elected = cluster.electLeader("n1");
        System.out.println("leader election for n1: " + elected);
        printState("cluster after election", cluster);

        boolean committed = cluster.leaderAppend("set x=10");
        System.out.println("\nappend set x=10 committed: " + committed);

        cluster.setAvailability("n4", false);
        cluster.setAvailability("n5", false);
        printState("\ncluster after two follower failures", cluster);

        committed = cluster.leaderAppend("set y=20");
        System.out.println("\nappend set y=20 committed with 3/5 up: " + committed);

        cluster.setAvailability("n3", false);
        printState("\ncluster after third failure", cluster);

        committed = cluster.leaderAppend("set z=30");
        System.out.println("\nappend set z=30 committed with 2/5 up: " + committed);
    }

    private static void printState(String label, RaftCluster cluster) {
        System.out.println(label);
        for (Map<String, Object> row : cluster.clusterState()) {
            System.out.println(
                "  "
                    + row.get("node")
                    + " avail="
                    + row.get("available")
                    + " role="
                    + row.get("role")
                    + " term="
                    + row.get("term")
                    + " log="
                    + row.get("log_len")
                    + " commit="
                    + row.get("commit_index")
            );
        }
    }
}
```

File: RaftCluster.java

```
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public class RaftCluster {
    enum Role {
        FOLLOWER,
        CANDIDATE,
        LEADER
    }

    static final class LogEntry {
        private final int term;
        private final String command;

        LogEntry(int term, String command) {
            this.term = term;
            this.command = command;
        }

        int getTerm() {
            return term;
        }

        String getCommand() {
            return command;
        }
    }

    static final class RaftNode {
        private final String nodeId;
        private Role role = Role.FOLLOWER;
        private int currentTerm = 0;
        private String votedFor = null;
        private final List<LogEntry> log = new ArrayList<>();
        private int commitIndex = -1;
        private boolean available = true;

        RaftNode(String nodeId) {
            this.nodeId = nodeId;
        }

        boolean requestVote(String candidateId, int term, int lastLogIndex, int lastLogTerm) {
            if (!available) {
                return false;
            }
            if (term < currentTerm) {
                return false;
            }
```

```
        if (term > currentTerm) {
            currentTerm = term;
            votedFor = null;
            role = Role.FOLLOWER;
        }
        if (votedFor != null && !votedFor.equals(candidateId)) {
            return false;
        }
        if (!isCandidateLogUpToDate(lastLogIndex, lastLogTerm)) {
            return false;
        }
        votedFor = candidateId;
        return true;
    }

    boolean appendEntries(
        int leaderTerm,
        int prevLogIndex,
        int prevLogTerm,
        List<LogEntry> entries,
        int leaderCommit
    ) {
        if (!available) {
            return false;
        }
        if (leaderTerm < currentTerm) {
            return false;
        }

        currentTerm = leaderTerm;
        role = Role.FOLLOWER;

        if (prevLogIndex >= 0) {
            if (prevLogIndex >= log.size()) {
                return false;
            }
            if (log.get(prevLogIndex).getTerm() != prevLogTerm) {
                return false;
            }
        }

        int insertAt = prevLogIndex + 1;
        while (log.size() > insertAt) {
            log.remove(log.size() - 1);
        }
        log.addAll(entries);

        if (leaderCommit > commitIndex) {
            commitIndex = Math.min(leaderCommit, log.size() - 1);
        }

        return true;
```

```
        }

        private boolean isCandidateLogUpToDate(int lastLogIndex, int lastLogTerm) {
            int myLastIndex = log.size() - 1;
            int myLastTerm = myLastIndex >= 0 ? log.get(myLastIndex).getTerm() : -1;
            if (lastLogTerm != myLastTerm) {
                return lastLogTerm > myLastTerm;
            }
            return lastLogIndex >= myLastIndex;
        }
    }

    private final List<RaftNode> nodes;
    private String leaderId = null;

    public RaftCluster(List<String> nodeIds) {
        if (nodeIds.size() < 3) {
            throw new IllegalArgumentException("cluster should have at least 3 nodes");
        }
        nodes = new ArrayList<>();
        for (String nodeId : nodeIds) {
            nodes.add(new RaftNode(nodeId));
        }
    }

    public boolean electLeader(String candidateId) {
        RaftNode candidate = node(candidateId);
        if (!candidate.available) {
            return false;
        }

        candidate.role = Role.CANDIDATE;
        candidate.currentTerm += 1;
        candidate.votedFor = candidate.nodeId;
        int term = candidate.currentTerm;
        int votes = 1;

        int lastLogIndex = candidate.log.size() - 1;
        int lastLogTerm = lastLogIndex >= 0 ? candidate.log.get(lastLogIndex).getTerm() : -1;

        for (RaftNode node : nodes) {
            if (node.nodeId.equals(candidateId)) {
                continue;
            }
            if (node.requestVote(candidateId, term, lastLogIndex, lastLogTerm)) {
                votes += 1;
            }
        }

        if (votes >= majority()) {
            for (RaftNode node : nodes) {
                node.role = Role.FOLLOWER;
```

```
                if (node.currentTerm < term) {
                    node.currentTerm = term;
                }
            }
            candidate.role = Role.LEADER;
            leaderId = candidateId;
            return true;
        }

        candidate.role = Role.FOLLOWER;
        return false;
    }

    public boolean leaderAppend(String command) {
        RaftNode leader = leader();
        if (leader == null || !leader.available) {
            return false;
        }

        LogEntry entry = new LogEntry(leader.currentTerm, command);
        leader.log.add(entry);
        int entryIndex = leader.log.size() - 1;
        int prevIndex = entryIndex - 1;
        int prevTerm = prevIndex >= 0 ? leader.log.get(prevIndex).getTerm() : -1;

        int acknowledgements = 1;
        for (RaftNode node : nodes) {
            if (node.nodeId.equals(leader.nodeId)) {
                continue;
            }
            if (node.appendEntries(leader.currentTerm, prevIndex, prevTerm, List.of(entry), leader.c
ommitIndex)) {
                acknowledgements += 1;
            }
        }

        if (acknowledgements >= majority()) {
            leader.commitIndex = entryIndex;
            for (RaftNode node : nodes) {
                if (node.nodeId.equals(leader.nodeId)) {
                    continue;
                }
                node.appendEntries(leader.currentTerm, entryIndex, entry.getTerm(), List.of(), leade
r.commitIndex);
            }
            return true;
        }

        leader.log.remove(leader.log.size() - 1);
        return false;
    }
```

```java
    public void setAvailability(String nodeId, boolean available) {
        RaftNode node = node(nodeId);
        node.available = available;
        if (!available && nodeId.equals(leaderId)) {
            leaderId = null;
        }
    }

    public List<Map<String, Object>> clusterState() {
        List<Map<String, Object>> rows = new ArrayList<>();
        for (RaftNode node : nodes) {
            Map<String, Object> row = new LinkedHashMap<>();
            row.put("node", node.nodeId);
            row.put("available", node.available);
            row.put("role", node.role.toString());
            row.put("term", node.currentTerm);
            row.put("log_len", node.log.size());
            row.put("commit_index", node.commitIndex);
            rows.add(row);
        }
        return rows;
    }

    private RaftNode leader() {
        if (leaderId == null) {
            return null;
        }
        RaftNode node = node(leaderId);
        if (node.role != Role.LEADER) {
            return null;
        }
        return node;
    }

    private RaftNode node(String nodeId) {
        for (RaftNode node : nodes) {
            if (node.nodeId.equals(nodeId)) {
                return node;
            }
        }
        throw new IllegalArgumentException("unknown node: " + nodeId);
    }

    private int majority() {
        return (nodes.size() / 2) + 1;
    }
}
```